



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»
РТУ МИРЭА

ДИСЦИПЛИНА	Программирование управляемых технических систем (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	ПТИП (укажите название учебного института)
КАФЕДРА	Индустриального программирования (укажите полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Материалы для практических занятий (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Закожурников Сергей Сергеевич (фио)
СЕМЕСТР	2, 2026 (указать семестр обучения, учебный год)

Практическая работа

Изучение правил. Создание дашбордов.

Неявные переменные внутри выполняемого блока

Помимо неявно доступных переменных для элементов и команд / состояний, правила могут иметь дополнительные предварительно определенные переменные в зависимости от их триггеров:

- `receivedCommand` — неявно доступно в каждом правиле, имеющем хотя бы один триггер командного события;
- `previousState` — неявно доступно в каждом правиле, имеющем хотя бы один триггер события изменения статуса;
- `newState` — неявно доступно в каждом правиле, имеющем хотя бы один триггер события обновления или изменения статуса;
- `triggeringItemName` — неявно доступно в каждом правиле, которое имеет хотя бы одно обновление статуса, изменение статуса или триггер командного события;
- `triggeringItem` — неявно доступен в каждом правиле, имеющем триггер «Член в»;
- `receivedEvent` — неявно доступно в каждом правиле с канальным триггером.

Раннее возвращение

Можно досрочно вернуться из правила, не выполняя остальные инструкции, подобные описанному в таблице 25.

Таблица 25

```
if (Temperature.state > 20) {  
    return;  
}  
Heating.sendCommand(ON)
```

Предостережение: обратите внимание на точку с запятой после оператора `return`, который завершает команду без дополнительного аргумента.

Защита конкуренции

Если правило срабатывает при событиях пользовательского интерфейса, может потребоваться защита от параллелизма. Это описано в таблице 26.

Таблица 26

```
import java.util.concurrent.locks.ReentrantLock  
  
val ReentrantLock lock = new ReentrantLock()  
  
rule ConcurrentCode  
when  
    Item Dummy received update  
then  
    lock.lock()  
    try {
```

```

        // выполнение действий
    } finally {
        lock.unlock()
    }
end

```

Трансформации

Сервисы openHAB Transformation могут использоваться в правилах для преобразования / перевода / преобразования данных. Общий синтаксис выглядит следующим образом: `transform("<transformation-identifier>", "<transf. expression or transf. file name>", <input-data or variable>)`, где:

- `<transformation-identifier>` — Сокращенный идентификатор услуги трансформации
- `<transf. expression or transf. file name>` — Специальная услуга трансформации
- `<input-data or variable>` — Данные для преобразования ДОЛЖНЫ иметь тип данных *String*.

Примеры трансформаций приведены в таблице 27 ниже.

Таблица 27

```

var condition = transform("MAP", "window_esp.map", "CLOSED")
var temperature = transform("JSONPATH", "$.temperature", jsonstring)
var fahrenheit = transform("JS", "convert-C-to-F.js", temperature)

```

Метод `transform` пытается преобразовать заданное значение, и, если это не удастся, это возвращает исходное значение неизменным. В отличие от `transform` метода, `transformRaw` метод не улавливает никакие `TransformationException` символы внутри себя, а пропускает их, так что вызывающий может обработать их. Синтаксис следующий: `transformRaw("<transformation-identifier>", "<transf. expression or transf. file name>", <input-data or variable>)`. Примеры показаны в таблице 28.

Таблица 28

```

try {
    var temperature = transformRaw("JSONPATH", "$.temperature",
    jsonstring)
}
catch(TransformationException e) {
    logError("Error", "Я написал какое-то кривое правило: " +
    e.getMessage)
}
finally {
    // всегда запускается даже если произошла ошибка! Можно использовать для
    очистки
}

```

Запись журналов

Вы можете создавать сообщения журнала из ваших правил, чтобы облегчить отладку. Есть несколько методов ведения журнала, доступных из ваших правил, подписи Java, описанные в таблице 27.

Таблица 27

```
logDebug(String loggerName, String format, Object... args)
logInfo(String loggerName, String format, Object... args)
logWarn(String loggerName, String format, Object... args)
logError(String loggerName, String format, Object... args)
```

В каждом случае `loggerName` параметр объединяется со строкой `org.openhab.core.model.script` для создания имени регистратора `log4j`. Например, если ваш файл правил содержит следующее сообщение журнала: `logDebug("kitchen", "Kitchen light turned on")`, для того что бы сообщения появлялись на консоли можно сделать следующие настройки журнала: `log:set DEBUG org.openhab.core.model.script.kitchen.`

Примеры правил

Ниже, в таблице 28, приведены некоторые примеры общих правил.

Таблица 28

```
var Number counter

// установка начальных значений счётчиков
// хотя можно это и в объявлении переменной делать
rule "Поехали"
when
    System started
then
    counter = 0
end

// Увеличиваем счётчик в полночь
rule "а счётчик-то тикает"
when
    Time cron "0 0 0 * * ?"
then
    counter = counter + 1
end

// в полдень доложить о количестве суток автономного полёта!
rule "Сообщить сколько дней стоит"
when
    Time is noon or
    Item AnnounceButton received command ON
then
```

```

    say("Всё хорошо уже " + counter + " дней")
end

// меняет счётчик по команде от начальства
rule "Улучшить счётчик"
when
    Item SetCounterItem received command
then
    counter = receivedCommand as DecimalType
end

// включает свет, когда один из нескольких датчиков движения получил команду
ВКЛ, выключает его, когда один из нескольких полученных команд ВЫКЛ.
rule "Прожектор в лицо"
when
    Member of MotionSensors received command
then
    if(receivedCommand == ON) Light.sendCommand(ON)
    else Light.sendCommand(OFF)
end

rule "Побудка на рассвете"
when
    Channel "astro:sun:home:rise#event" triggered
then
    switch(receivedEvent) {
        case "START": {
            Light.sendCommand(ON)
        }
    }
end

```

Проектирование интерфейсов дашбордов с помощью НАВPanel

Пользовательский интерфейс НАВPanel устанавливается по умолчанию при выборе любого пакета первоначальной установки и позволяет создавать удобные информационные панели, особенно подходящие для (например, настенных) планшетов. Эти информационные панели можно проектировать в интерактивном режиме во встроенном конструкторе, а не с помощью файлов конфигурации.

Несмотря на схожесть, информационные панели и карты сайта

НАВPanel представляют собой отдельные концепции и могут разрабатываться независимо друг от друга, поскольку они не связаны друг с другом; однако они полагаются и воздействуют на элементы, которые поэтому должны быть определены в первую очередь. Демонстрационный установочный пакет, доступный для установки при первом запуске openHAB, определяет ряд элементов-примеров и настраивает НАВPanel с полным набором информационных панелей для демонстрации возможного конечного результата. Он такой же, как установленный на демонстрационном сервере openHAB, и его можно модифицировать, не рискуя ничего сломать: это лучшая площадка для знакомства с функциями НАВPanel.

Концепции

НАВPanel имеет собственную терминологию сущностей (рис. 3)

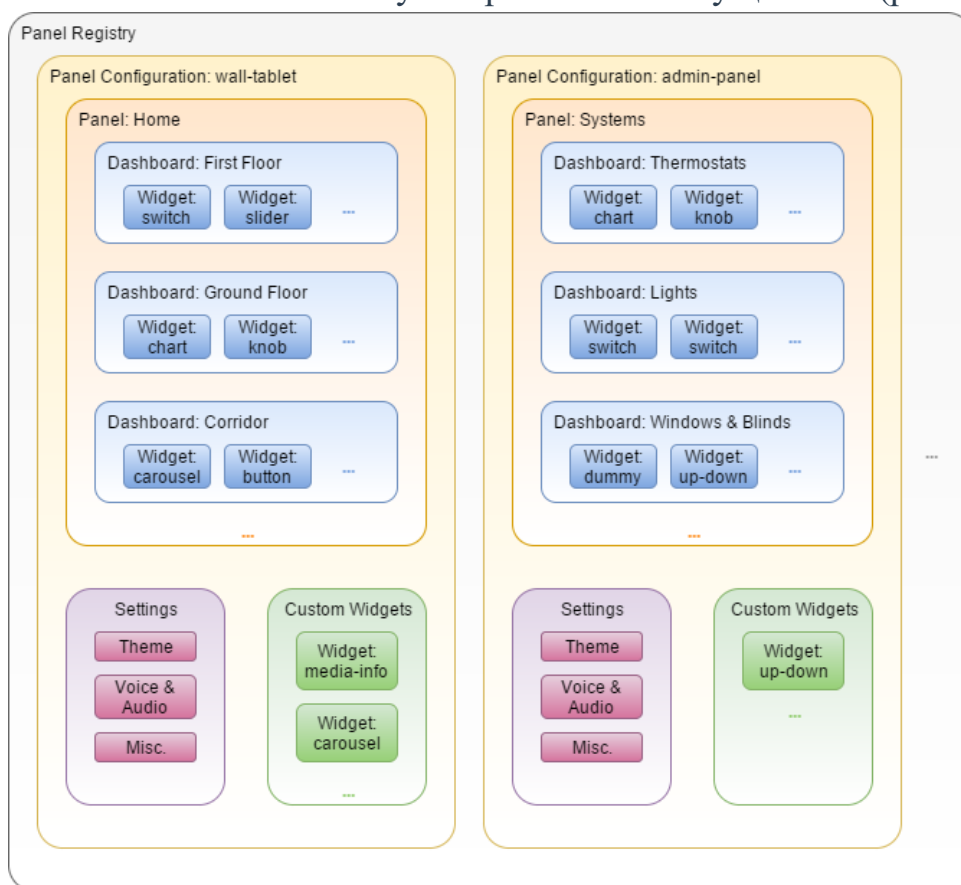


Рисунок 3 – Сущности HabPanel

Реестр **Panel** — это центральное хранилище, используемое НАВPanel в данном экземпляре openHAB, он содержит несколько **конфигураций Panel** ;

Конфигурация **панели** — это контейнер, содержащий **панель** вместе с ее **настройками** и определением **пользовательских виджетов**. Каждое устройство, на котором работает НАВPanel (настольный браузер, планшет...), имеет активную конфигурацию панели и отображает связанную с ней панель;

Панель — это набор **информационных панелей** (или страниц), которые могут быть представлены конечным пользователям, которые также могут легко переключаться между ними с помощью меню.

Информационная панель состоит из отдельных **виджетов**, размещенных на поверхности панели во время разработки. Есть несколько

типов встроенных *стандартных виджетов*, настраиваемых отдельно, а также администратор может разрабатывать (или импортировать) *пользовательские виджеты*.

О сохранении данных

По умолчанию при запуске HABPanel в новом браузере или на новом устройстве будет отображаться руководство, позволяющее пользователю начать с нуля или переключиться на ранее определенную конфигурацию панели. **Пока не будет создана (или выбрана) конфигурация панели, HABPanel будет работать в режиме «локального хранилища» для этого устройства: настройки будут сохраняться только в локальном хранилище браузера, и ничего не будет сохраняться на сервере.**

В отличие от этого, когда установлена активная конфигурация панели, каждое изменение, выполненное на устройстве, будет обновлять конфигурацию панели на сервере. Это позволяет совместно использовать конфигурацию панели между устройствами, потому что другие браузеры и устройства, использующие эту конфигурацию панели, увидят изменения при обновлении страницы — это полезно, например, для удобного создания панели на компьютере, а затем использования ее на планшете.

Чтобы переключиться с локального хранилища на конфигурацию панели, размещенной на сервере, перейдите на страницу настроек из главного меню или боковой панели (см. ниже). Список конфигураций панели будет представлен в разделе *«Текущая конфигурация хранилища»*; если доступна только опция *«Локальное хранилище»*, **нажмите на ссылку «Сохранить текущую конфигурацию» в новой конфигурации панели**, дайте ей имя для идентификации (избегайте пробелов или специальных символов), и она должна быть добавлена в список. Радиокнопка также автоматически проверяется, что означает, что теперь она является активной конфигурацией панели.

Даже при наличии активной конфигурации панели HABPanel использует хранилище браузера для синхронизации локально управляемой копии. С помощью ссылки **«Изменить конфигурацию локальной панели (только для экспертов) под параметром конфигурации» хранилища «Локальное хранилище»** на экране настроек необработанную структуру конфигурации панели можно проверить, изменить и экспортировать или импортировать из/в файл.json. Это также альтернативный способ резервного копирования, восстановления и совместного использования конфигурации.

HABPanel использует переменные конфигурации службы для хранения своих данных на сервере openHAB. Доступ к ним можно получить с помощью пользовательского интерфейса Paper (*Configuration > Services > UI > HABPanel > Configure*) или в консоли openHAB Karaf:

```
openhabs> config:edit org.openhab.habpanel
openhabs> config:property-get <
```

Определены следующие свойства:

- `panelsRegistry`: содержит весь реестр, сериализованный в JSON, он поддерживается приложением и не должен изменяться напрямую (редактирование вручную, хотя и возможно, настоятельно не рекомендуется);
- `lockEditing`: при включении все экземпляры `HABPanel` будут скрывать свои функции редактирования (необходимо обновление страницы). Когда панели готовы и стабильны, рекомендуется включить этот параметр, чтобы конечные пользователи не могли легко изменить их;
- `initialPanelConfig`: если этот параметр не установлен, и предыдущая локальная конфигурация не обнаружена, учебник будет отображаться до тех пор, пока не будут добавлены некоторые панели мониторинга или не будет выбрана конфигурация панели. Этот параметр позволяет обойти обучение и переключиться непосредственно на существующую конфигурацию панели с заданным именем.

Задание:

1. Создать неявную переменную
2. Создать правило с ранним возвращением
3. Создать правило с защитой от параллелизма
4. Создать сервис `Transformation`
5. Создать журнал из правил
6. Создать дашборд с помощью `HABPanel`